

Chapter 6

Backtracking Techniques

Definition 6.0.1

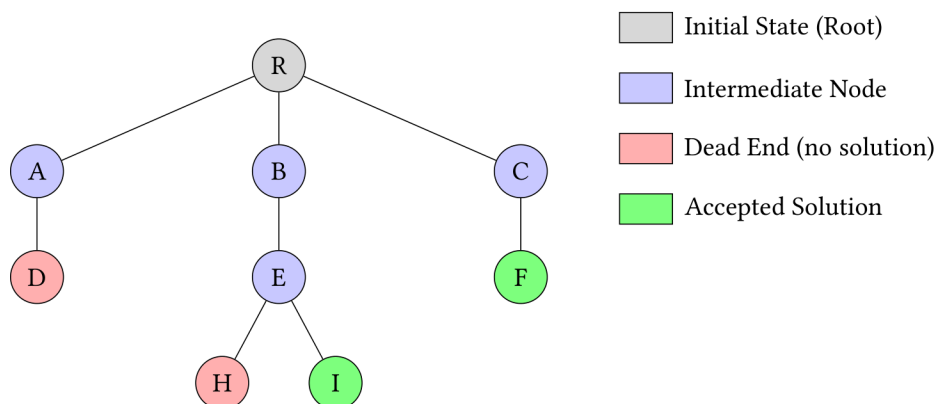
Backtracking is a problem-solving algorithmic technique that involves finding a solution incrementally by trying different options and undoing them if they lead to a dead end.

Backtracking is a systematic, refined form of brute-force search. It constructs candidate solutions step-by-step and abandons (“backtracks” from) a partial candidate as soon as it determines that the candidate cannot possibly be completed to a valid solution.

6.0.1 How Backtracking Works

The backtracking process can be visualized as a depth-first traversal of a **state-space tree**:

- **Initial State (root):** The starting empty partial solution.
- **Intermediate Nodes:** Partial solutions still being explored.
- **Dead Ends:** Nodes that violate constraints; the algorithm *backtracks* from these.
- **Goal/Accepted Nodes:** Complete valid solutions.



1. **Choose:** Pick an option from the available choices at the current state.
2. **Explore:** Recursively try to extend the partial solution with this choice.
3. **Check Validity:** If the partial solution violates any constraint, abandon it.
4. **Backtrack:** If the choice leads to a dead end, undo it and try another option.
5. **Success:** If a complete valid solution is built, record/return it.

6.0.2 Applications

Backtracking is mainly used for problems where solutions must satisfy a set of constraints (Constraint Satisfaction Problems (CSPs)):

- Sudoku
- N-Queens problem
- Tic-tac-toe and Chess
- Graph Coloring
- Hamiltonian Cycle
- Sum of Subsets, 0/1 Knapsack

6.0.3 Advantages of Backtracking

- Backtracking stops exploring as soon as a path violates constraints — **faster than pure brute force**.
- It explores all possibilities in a **structured way** (Depth-First Search).
- If a solution exists in the state-space tree, backtracking **will find it** (guaranteed).
- Most backtracking implementations are easy to **implement using recursion**.
- It works very well for **Constraint Satisfaction Problems**.

6.0.4 Disadvantages of Backtracking

- **High time complexity** (often exponential).
- Performance **degrades rapidly** with increasing input size.
- **It does not guarantee optimal solution** (unless all solutions are explored).
- It can be inefficient if the state-space tree is large and has many dead ends.
- Without heuristics, it may explore many unnecessary paths before finding a solution.

6.0.5 Recursion vs Backtracking

While backtracking is implemented using recursion, the two are conceptually distinct.

Recursion	Backtracking
A programming technique where a function calls itself to solve smaller subproblems.	An <i>algorithmic technique</i> that solves problems by trying different possibilities and undoing wrong choices.
General problem-solving method.	Specialized technique built on recursion for constraint-based problems.
Solves problems by breaking them into smaller, similar subproblems.	Solves problems with multiple choices by systematically exploring options and reversing when needed.
Does not necessarily explore multiple alternatives.	Always explores alternatives and revokes decisions.
Examples: Tree/Graph traversal, Towers of Hanoi, Merge Sort, Quick Sort, Binary Search, Divide & Conquer.	Examples: N-Queen, Rat in a Maze, Knight's Tour, Sudoku Solver, Graph Coloring, Sum of Subsets.

Note:-

Every backtracking algorithm is recursive, but not every recursive algorithm is backtracking. Backtracking = Recursion + undoing choices when constraints fail.

6.1 N-Queens Problem

6.1.1 Problem Definition

Definition 6.1.1

Given an $N \times N$ chessboard, place N queens on the board such that *no two queens attack each other*, i.e., no two queens share the same row, column, or diagonal.

The N-Queens problem is the **classical example of backtracking**.

6.1.2 Constraints

A placement of a queen at (row_i, col_i) conflicts with another queen at (row_j, col_j) iff:

Same row: $row_i = row_j$ and **Same column:** $col_i = col_j$ and **Same diagonal:** $|row_i - row_j| = |col_i - col_j|$

6.1.3 Algorithm

Algorithm 6.1.1

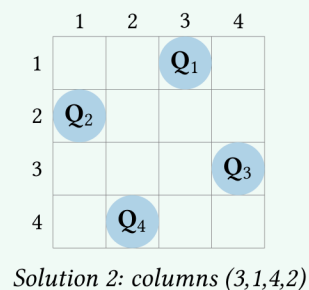
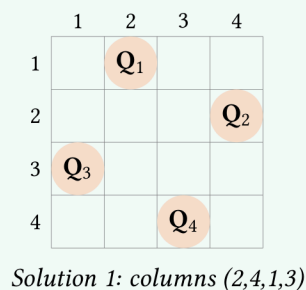
1. Place a queen in the first column of the first row.
2. Check if placing the queen conflicts with any previously placed queens.
3. If no conflict, proceed to the next row. If conflict, try placing in the next column.
4. Repeat for all rows.
5. If all queens are placed without conflict, a solution is found.
6. If no solution is possible from the current state, **backtrack** to the previous row.

Example 6.1.1

Problem : Given 4×4 chessboard, arrange four queens in a way, such that no two queens attack each other. That is, no two queens are placed in the same row, column, or diagonal.

Step-by-step solution: For a 4×4 board, there are exactly **two solutions**.

Solutions

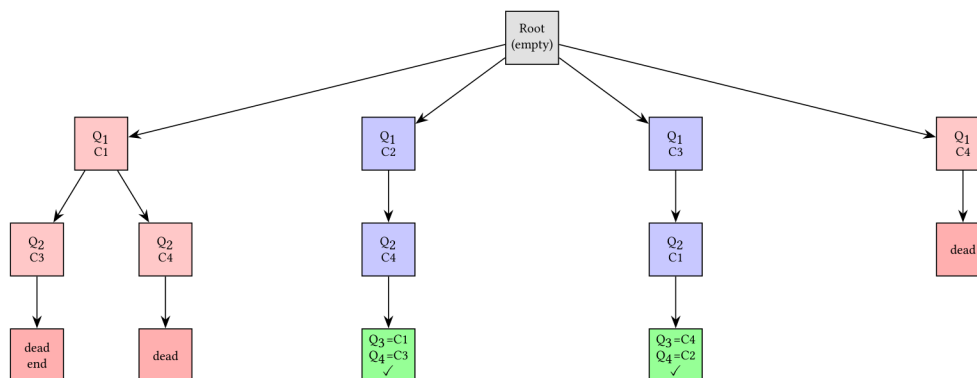


Step-by-Step Trace (Solution 1)

Step	Action	Board State (col positions)
1	Place Q_1 at (Row 1, Col 1)	(1, -, -, -)
2	Try Q_2 at (R2,C1) $\times$ same col; (R2,C2) $\times$ diag; (R2,C3) $\checkmark$	(1, 3, -, -)
3	Try Q_3 at (R3,C1) $\times$; (R3,C2) $\times$; (R3,C3) $\times$; (R3,C4) $\times$ all conflict	Backtrack to Q_2
4	Move Q_2 to (R2, C4)	(1, 4, -, -)
5	Try Q_3 at (R3,C2) $\checkmark$	(1, 4, 2, -)
6	Try Q_4 at all columns – all conflict	Backtrack to Q_1
7	Move Q_1 to (R1, C2)	(2, -, -, -)
8	Place Q_2 at (R2, C4), Q_3 at (R3, C1), Q_4 at (R4, C3)	(2, 4, 1, 3) $\checkmark$

Step-by-Step Trace (Solution 2)

Step	Action	Board State (col positions)
1	Backtrack from Solution 1; move Q_1 to (R1, C3)	(3, -, -, -)
2	Try Q_2 at (R2,C1) $\checkmark$ (not same col, not diagonal)	(3, 1, -, -)
3	Try Q_3 at (R3,C1) $\times$ same col as Q_2 ; (R3,C2) $\times$ diag with Q_2 ; (R3,C3) $\times$ same col as Q_1 ; (R3,C4) $\checkmark$	(3, 1, 4, -)
4	Try Q_4 at (R4,C1) $\times$ same col as Q_2 ; (R4,C2) $\checkmark$ (safe from all)	(3, 1, 4, 2) $\checkmark$
5	Continue exploring: backtrack and try Q_1 at (R1, C4)	(4, -, -, -)
6	Try Q_2 at all columns – all conflict or lead to dead ends	Backtrack
7	All branches exhausted – algorithm terminates	Done

State-Space Tree for 4-Queens (Schematic)

6.1.4 Time Complexity

In the worst case:

- For the first row, try placing a queen in all n columns.
- For the second row, with one queen placed, try at most $n - 1$ columns.
- For the third row, with two queens placed, try at most $n - 2$ columns.
- And so on . . .

The total number of possibilities explored:

$$T(n) \approx n \cdot (n - 1) \cdot (n - 2) \cdots 1 = n!$$

$$T(n) = O(n!)$$

6.1.5 Space Complexity

The algorithm uses:

- The recursion depth can reach n , so the recursion stack requires: $O(n)$
- Arrays to track occupied **columns**, **left diagonals**, and **right diagonals** — each $O(n)$.

Total:

$$O(n) + O(n) + O(n) = O(n)$$

$$S(n) = O(n!)$$

If the full board is stored

$$O(n^2) \quad (\text{an } n \times n \text{ matrix has } n^2 \text{ cells})$$

6.2 Sum of subsets

6.2.1 Problem Definition

Definition 6.2.1

Given a set of n positive integers $W = \{w_1, w_2, \dots, w_n\}$ and a positive integer M , find all subsets of W whose sum equals M .

Mathematically, find binary vector $(x_1, x_2, \dots, x_n)$ such that

$$\sum_{i=1}^n w_i \cdot x_i = M, \quad x_i \in \{0, 1\}$$

where,

- $x_i = 1 \Rightarrow$ include w_i in the subset
- $x_i = 0 \Rightarrow$ exclude w_i from the subset

Precondition: The weights are sorted in ascending order: $w_1 < w_2 < w_3 < \dots < w_n$.

6.2.2 Algorithm

Algorithm 6.2.1

SUB_SET_PROBLEM(i , sum , W , M , remSum)

i : current item index

sum : sum of selected items so far and **remSum** : sum of remaining items

M : target sum and **W** : set of weights

X : solution vector

1. **if** **FEASIBLE_SUB_SET**(i , sum , M , remSum) = 0 **then: return**
2. **if** $\text{sum} = M$ **then**
 - print $X[1..i]$ *// solution found*
 - return
3. **else**
 - // Include current item*
 - $X[i + 1] \leftarrow 1$
 - SUB_SET_PROBLEM**($i + 1$, $\text{sum} + w[i + 1]$, M , W , $\text{remSum} - w[i + 1]$)
 - // Exclude current item*
 - $X[i + 1] \leftarrow 0$
 - SUB_SET_PROBLEM**($i + 1$, sum , M , W , $\text{remSum} - w[i + 1]$)

FEASIBLE_SUB_SET(i , sum , M , remSum):

1. **if** $\text{sum} + \text{remSum} < M$ **or** $\text{sum} > M$ **then**
 - return** 0 *// not feasible*
2. **else**
 - return** 1 *// feasible*

Bounding Conditions (Pruning)

The feasibility function prunes branches using two checks:

1. **Over-shoot:** If $\text{sum} > M$, the current subset already exceeds the target $\Rightarrow$ prune.
2. **Under-shoot:** If $\text{sum} + \text{remSum} < M$, even including all remaining items cannot reach $M \Rightarrow$ prune.

6.2.3 Time Complexity

For every element we have two choices: *include* or *exclude*.

Level	Nodes
0	1
1	2
2	2^2
$\vdots$	$\vdots$
i	2^i
n	2^n

Total nodes in the state-space tree (without pruning):

$$T(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^n = 2^{n+1} - 1$$

$$T(n) = O(2^n)$$

This is the worst-case complexity. The feasibility condition may prune some branches, but in the worst case, pruning may not significantly reduce the search tree.

6.2.4 Space Complexity

1. **Solution vector X :** A 1D array of size n holds the include/exclude decision for each item $\Rightarrow O(n)$.
2. **Recursion stack:** The maximum depth of recursion is n , because one item is processed at each level: $\Rightarrow O(n)$.

$$S(n) = O(n)$$

This space complexity assumes that solutions are printed immediately and the complete state-space tree is not stored. If all solutions are stored, additional exponential output space may be required.

Example 6.2.1

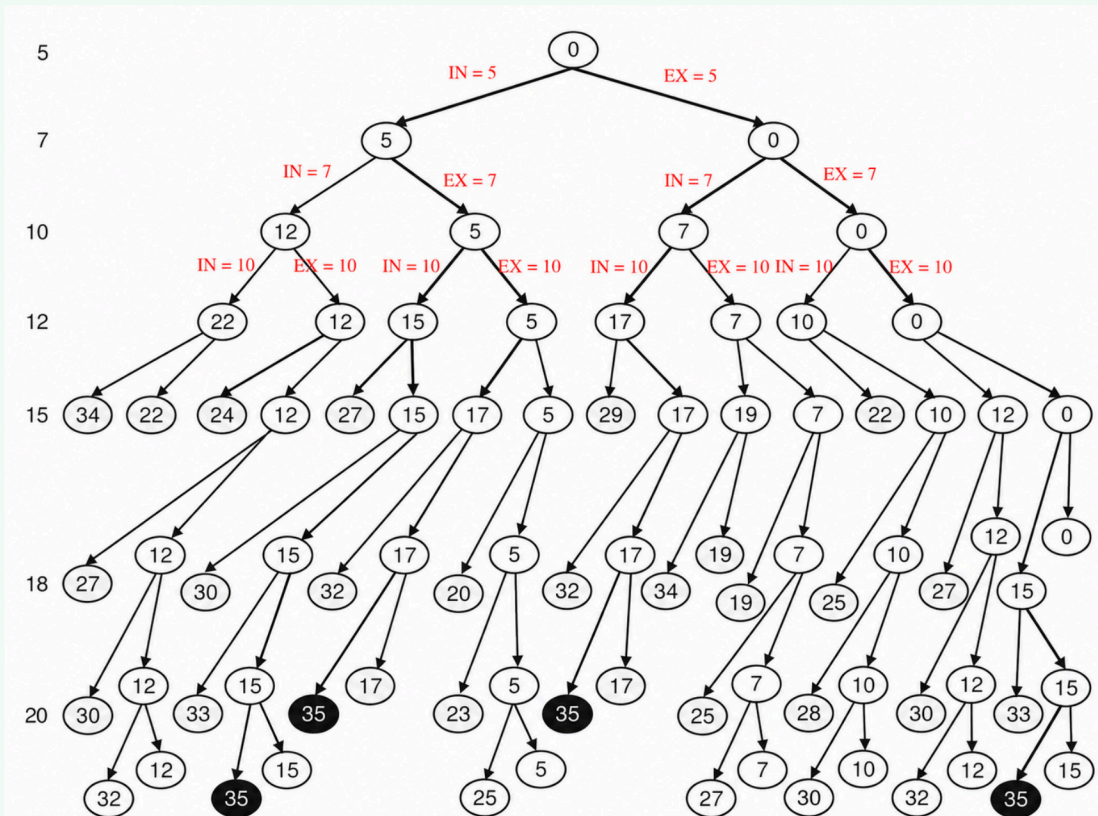
Solve the sum of subset problems using backtracking algorithmic strategy for the following data: $n = 7$ and $W = (w_1, w_2, w_3, w_4, w_5, w_6, w_7) = 5, 7, 10, 12, 15, 18, 20$ and $M = 35$.

Solution: There may be multiple solutions. A state-space tree for the above sequence is shown here: The number in the leftmost column shows the element under consideration. The left and right branches in the tree indicate inclusion and exclusion of the corresponding element at that level, respectively.

The black circle is the solution state.

We get the four solutions:

(5, 10, 20)
 (5, 12, 18)
 (7, 10, 18)
 (15, 20)



The trace below shows how the algorithm explores subsets, backtracking whenever the sum exceeds $M = 35$.

Step	Subset	Sum Condition	Action
1	{ }	0	Initial condition
2	{5}	$5 < 35$	Select 5 and add next element
3	{5, 7}	$12 < 35$	Select 7 and add next element
4	{5, 7, 10}	$22 < 35$	Select 10 and add next element
5	{5, 7, 10, 12}	$34 < 35$	Select 12 and add next element
6	{5, 7, 10, 12, 15}	$49 > 35$	Sum exceeds M, backtrack remove 15
7	{5, 7, 10, 12}	$34 < 35$	Exclude 15 and add next element
8	{5, 7, 10, 12, 18}	$52 > 35$	Backtrack remove 18
9	{5, 7, 10, 12}	$34 < 35$	Exclude 18 and add next element
10	{5, 7, 10, 12, 20}	$54 > 35$	Backtrack remove 20
11	{5, 7, 10, 12}	$34 < 35$	No more elements, backtrack remove 12
12	{5, 7, 10}	$22 < 35$	Exclude 12 and add next element
13	{5, 7, 10, 15}	$37 > 35$	Backtrack remove 15
14	{5, 7, 10}	$22 < 35$	Exclude 15 and add next element
15	{5, 7, 10, 18}	$40 > 35$	Backtrack remove 18
16	{5, 7, 10}	$22 < 35$	Exclude 18 and add next element
17	{5, 7, 10, 20}	$42 > 35$	Backtrack remove 20
18	{5, 7, 10}	$22 < 35$	No more elements, backtrack remove 10
19	{5, 7}	$12 < 35$	Exclude 10 and add next element
20	{5, 7, 12}	$24 < 35$	Select 12 and add next element
21	{5, 7, 12, 15}	$39 > 35$	Backtrack
22	{5, 7, 12}	$24 < 35$	Exclude 15 and add next element

Step	Subset	Sum Condition	Action
23	{5, 7, 12, 18}	$42 > 35$	Backtrack
24	{5, 7, 12}	$24 < 35$	Exclude 18 and add next element
25	{5, 7, 12, 20}	$44 > 35$	Backtrack
26	{5, 7}	$12 < 35$	Backtrack remove 12
27	{5, 7}	$12 < 35$	Exclude 7 and add next element
28	{5, 10}	$15 < 35$	Select 10 and add next element
29	{5, 10, 12}	$27 < 35$	Select 12 and add next element
30	{5, 10, 12, 15}	$42 > 35$	Backtrack
31	{5, 10, 12}	$27 < 35$	Exclude 15 and add next element
32	{5, 10, 12, 18}	$45 > 35$	Backtrack
33	{5, 10, 12}	$27 < 35$	Exclude 18 and add next element
34	{5, 10, 12, 20}	$47 > 35$	Backtrack
35	{5, 10}	$15 < 35$	Exclude 12 and add next element
36	{5, 10, 15}	$30 < 35$	Select 15 and add next element
37	{5, 10, 15, 18}	$48 > 35$	Backtrack
38	{5, 10, 15}	$30 < 35$	Exclude 18 and add next element
39	{5, 10, 15, 20}	$50 > 35$	Backtrack
40	{5, 10}	$15 < 35$	Exclude 15 and add next element
41	{5, 10, 18}	$33 < 35$	Select 18 and add next element
42	{5, 10, 18, 20}	$53 > 35$	Backtrack
43	{5, 10, 18}	$33 < 35$	Exclude 20 → No more elements → Backtrack
44	{5, 10}	$15 < 35$	Exclude 18 and add next element
45	{5, 10, 20}	$35 = 35$	Solution Found
⋮ (continues exploring all branches) ⋮			
...	{5, 12, 18}	35	Solution Found
...	{7, 10, 18}	35	Solution Found
...	{15, 20}	35	Solution Found

Question 6.1: Assignment

1. Solve the sum of subset problems using backtracking algorithmic strategy for the following data: $n=4$ and $W=(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$ and $M=31$.

6.3 Graph Coloring Problem

Definition 6.3.1

Graph coloring refers to the problem of coloring the vertices of a graph in such a way that *no two adjacent vertices have the same color*. This is also called the **vertex coloring problem**.

The graph coloring problem can be viewed in two ways:

- **Decision problem:** “Given M colors and a graph G , is such a coloring scheme possible?”
- **Optimization problem:** “Given a graph G , find the minimum number of colors required to color G .”

6.3.1 Step-by-Step Procedure

Step 1. List all vertices and available colors.

Step 2. Assign color 1 to vertex 1.

Step 3. For each subsequent vertex v :

- Try color 1 first.
- If any neighbor of v already has this color, try the next color.
- Repeat until a safe color is found.

Step 4. If no color from $1 \dots m$ is valid for vertex v , **backtrack** to the previous vertex and increment its color.

Step 5. Continue until all vertices are colored or all possibilities are exhausted.

6.3.2 Algorithm

Algorithm 6.3.1

Input: A graph G with V vertices and m available colors.

Output: All valid colorings of the graph using at most m colors.

Procedure GRAPHCOLORING(G, m, i):

- If $i > V$:
 - Print COLOR[1...V]
 - Return
- For $c \leftarrow 1$ to m :
 - If ISAFE(G, i, c) is true:
 - COLOR[i] $\leftarrow c$
 - GRAPHCOLORING($G, m, i + 1$)
 - COLOR[i] $\leftarrow 0$ // backtrack

Function ISAFE(G, i, c):

- For $j \leftarrow 1$ to $i - 1$:
 - If Adjacent(i, j) = 1 and COLOR[j] = c :
 - Return false
- Return true

6.3.3 Time Complexity

For each of the V vertices, the algorithm may try all m available colors. Therefore, the maximum number of complete color assignments is:

$$m \times m \times \cdots \times m = m^V$$

The state-space tree contains:

$$T(V) = 1 + m + m^2 + \cdots + m^V$$

Using the geometric-series formula:

$$T(V) = \frac{m^{V+1} - 1}{m - 1}$$

Therefore, the number of states explored in the worst case is:

$$\boxed{O(m^V)}$$

For each color assignment, the `IsSAFE` function may check up to V previously colored vertices. Thus, its cost is $O(V)$.

Hence, the overall worst-case time complexity is:

$$\boxed{T(V) = O(V \cdot m^V)}$$

Backtracking may prune invalid branches early, but the worst-case time complexity remains exponential.

6.3.4 Space Complexity

The `COLOR[1 . . . V]` array stores the color assigned to each vertex and requires:

$$O(V)$$

The maximum depth of the recursive call stack is also V , because one vertex is colored at each recursion level:

$$O(V)$$

Therefore, the auxiliary space complexity is:

$$O(V) + O(V) = O(V)$$

Hence:

$$\boxed{S(V) = O(V)}$$

This excludes the storage required for the input graph. If an adjacency matrix is used, the total space complexity is $O(V^2)$. If an adjacency list is used, graph storage requires $O(V + E)$ space.

6.4 Hamiltonian Cycle

Definition 6.4.1

A **Hamiltonian cycle** of an undirected graph $G = (V, E)$ is a closed path that visits every vertex $v \in V$ *exactly once* and returns to the starting vertex. It does *not* need to include every edge of G .

Definition 6.4.2: Hamiltonian Cycle Problem

Given an undirected, connected graph $G = (V, E)$, determine whether a Hamiltonian cycle exists, and if so, find one.

The problem is categorised in two ways:

- **Decision problem:** “Given a path, is it a Hamiltonian cycle of G ?”
- **Optimisation problem:** “Find a Hamiltonian cycle in G if one exists.”

The Hamiltonian cycle problem belongs to the class **NP-complete**.

Formal Constraints

Let $V = \{v_0, v_1, \dots, v_{n-1}\}$ be the sequence of vertices in a candidate path. A valid Hamiltonian cycle must satisfy:

- C1. Adjacency:** Consecutive vertices v_i and v_{i+1} must share an edge, i.e. $(v_i, v_{i+1}) \in E$ for all $i = 0, 1, \dots, n-2$.
- C2. Closure:** The last vertex v_{n-1} must be adjacent to the first vertex v_0 , i.e. $(v_{n-1}, v_0) \in E$.
- C3. Uniqueness:** No vertex appears more than once in the path; $v_i \neq v_j$ for all $i \neq j$.

6.4.1 Algorithm

Algorithm 6.4.1

Hamiltonian(i)

Input: Current index i in the path array V

Output: Prints Hamiltonian cycle if found

1. For $j \leftarrow 2$ to n : // Try all candidate vertices.
2. $V[i] \leftarrow j$
3. If FEASIBLE(i):
4. If $i = n - 1$: // Path is complete (all n vertices added)
5. If Adjacent($V[n-1], V[0]$): // Edge exists back to starting vertex
6. Print $V[0 \dots n-1], V[0]$
7. Else:
8. HAMILTONIAN($i + 1$) // Recurse to fill next position

Algorithm 6.4.2**Feasible(*i*)****Input:** Current index *i* in the path array *V***Output:** true if *V*[*i*] is a valid next vertex, else false

1. $flag \leftarrow \text{true}$
2. For $j \leftarrow 0$ to $i - 1$:
 - (a) If $V[i] = V[j]$: // Must not be a duplicate vertex
 - i. $flag \leftarrow \text{false}$
3. If $\neg \text{Adjacent}(V[i - 1], V[i])$: // Must be connected to previous vertex
 - (a) $flag \leftarrow \text{false}$
4. Return $flag$

6.4.2 Time Complexity

Starting from a fixed vertex, the algorithm recursively tries all possible unvisited vertices to construct a Hamiltonian cycle.

In the worst case, the graph is highly connected (dense graph), so at each step many choices are available.

- At level 0: starting vertex 1 is fixed $\rightarrow$ 1 choice
- At level 1, there are at most $(n - 1)$ choices.
- At level 2, there are at most $(n - 2)$ choices.
- At level 3, there are at most $(n - 3)$ choices.
- ...

Thus, the total number of possible paths explored is:

$$1.(n - 1)(n - 2)(n - 3) \cdots 1$$

$$= (n - 1)!$$

Therefore, the worst-case time complexity is:

$$O(n!)$$

6.4.3 Space Complexity

The algorithm mainly uses recursion and a path array to store the current Hamiltonian path.

- The recursion depth can go up to n because one recursive call is made for each vertex.
- The path array *V* stores at most n vertices.

Thus:

$$\text{Recursion stack space} = O(n)$$

and

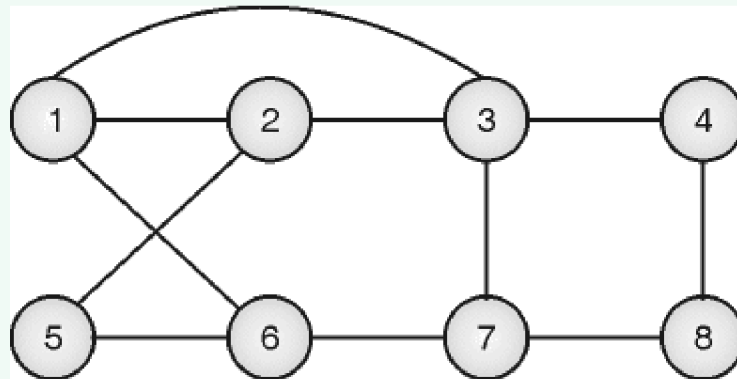
$$\text{Path storage space} = O(n)$$

Therefore, the overall space complexity is:

$$O(n)$$

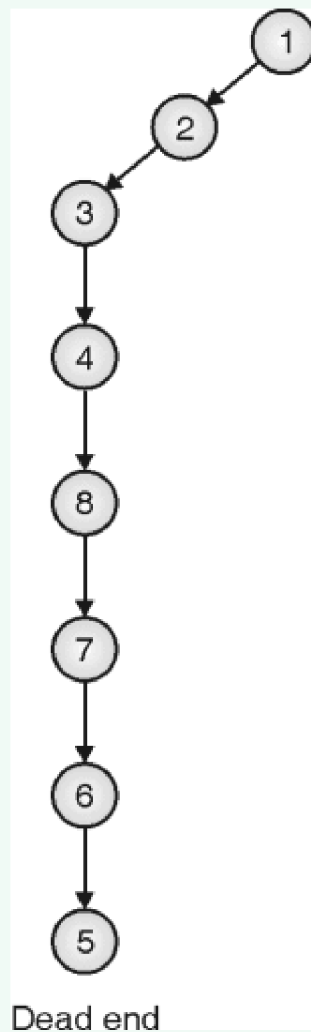
Example 6.4.1

Explain how to find a Hamiltonian Cycle using Backtracking in a given graph.

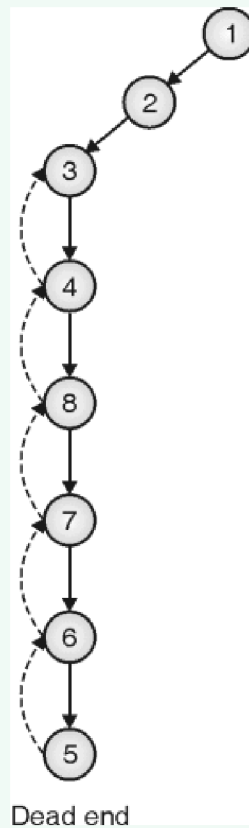


Solution: To find a Hamiltonian Cycle using backtracking, we can follow these steps:

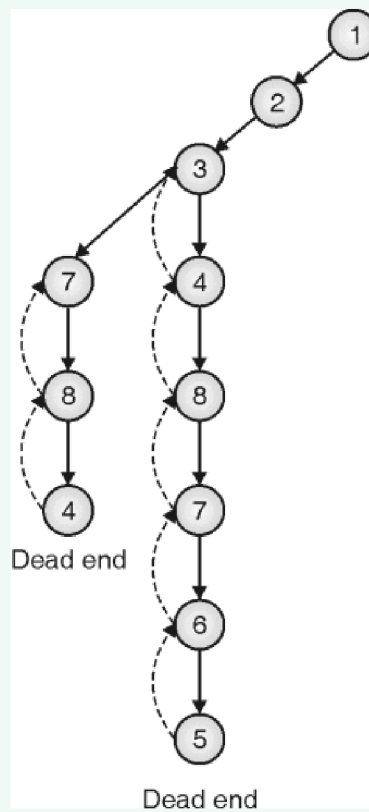
Step 1: Start from an arbitrary vertex (let's say vertex 1) and mark it as the first vertex in the path. The tour begins at vertex 1.



Step 2: There is no path from 5 to 1. So it's the dead-end state. Backtrack to the node from where the new path can be explored, that is 3 here.

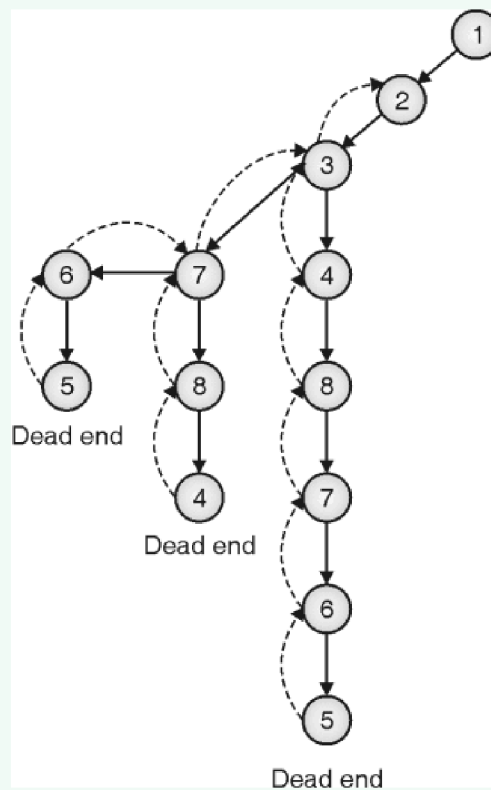


Step 3: New path also leads to a dead end so backtrack and explore all possible paths.

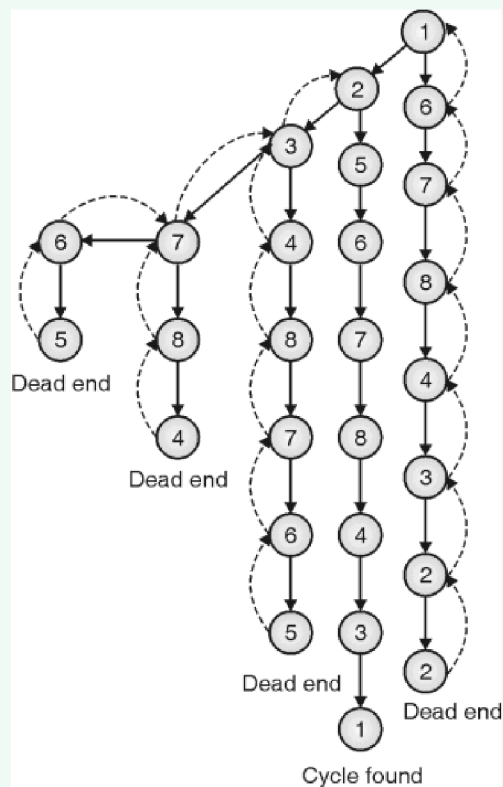


Step 4: Next path is also leading to a dead-end so keep backtracking until we get some node that can generate a

new path, i.e .vertex 2 here.



Step 5: One path leads to Hamiltonian cycle, next leads to a dead end so backtrack and explore all possible paths at each vertex.



Step 6: Total two Hamiltonian cycles are detected in a given graph.

6.5 0/1 Knapsack Problem (Backtracking)

6.5.1 Problem Definition

Definition 6.5.1

Given n items, each with a weight w_i and a profit p_i , and a knapsack of capacity W , select a subset of items such that

- Total weight $\leq W$, and
- Total profit is *maximized*.

Each item must be either fully included ($x_i = 1$) or fully excluded ($x_i = 0$) – no fractions.
Formally, maximize

$$\sum_{i=1}^n p_i x_i \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

6.5.2 Algorithm

Algorithm 6.5.1

Input:

- n – number of items
- W – maximum capacity
- $w[1..n]$ – weights
- $p[1..n]$ – profits

Output:

- maxProfit – maximum profit
- bestSet – items included in optimal solution

Initial Call: Knapsack(1, 0, 0, empty set) $\text{maxProfit} \leftarrow 0$

$\text{bestSet} \leftarrow []$

Procedure Knapsack($i, cw, cp, \text{currentSet}$):

1. If $cw \leq W$ and $cp > \text{maxProfit}$:
 - (a) $\text{maxProfit} \leftarrow cp$
 - (b) $\text{bestSet} \leftarrow \text{copy of currentSet}$
2. If $i = n$: (all items considered)
 - (a) Return
3. If $cw + w[i] \leq W$: (// Include item i (if it doesnot exceed capacity))
 - (a) Knapsack($i + 1, cw + w[i], cp + p[i], \text{currentSet} \cup \{i\}$)
4. Knapsack($i + 1, cw, cp, \text{currentSet}$) (// Exclude item i)

Return $\text{maxProfit}, \text{bestSet}$

6.5.3 Time Complexity

For each item, the algorithm has two possible choices:

- Include the item in the knapsack.
- Exclude the item from the knapsack.

Therefore, the recursion tree is a binary tree. For n items, the number of nodes at each level is:

$$1, 2, 2^2, 2^3, \dots, 2^n$$

Hence, the total number of nodes explored is:

$$T(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^n$$

Using the geometric series formula,

$$1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$$

Therefore,

$$T(n) = 2^{n+1} - 1$$

Ignoring constants and lower-order terms, the worst-case time complexity is:

$$O(2^n)$$

The algorithm has exponential time complexity because each of the n items can either be included or excluded. Thus, it may examine all 2^n possible subsets of items in the worst case.

6.5.4 Space Complexity

The algorithm uses recursion. At most, the recursion depth reaches n because one recursive call is made for each item. i.e The program does NOT keep the entire tree in memory. At one moment, only one path is active.

Thus, the stack space required is:

$$O(n)$$

Additionally:

- currentSet stores at most n items
- bestSet also stores at most n items

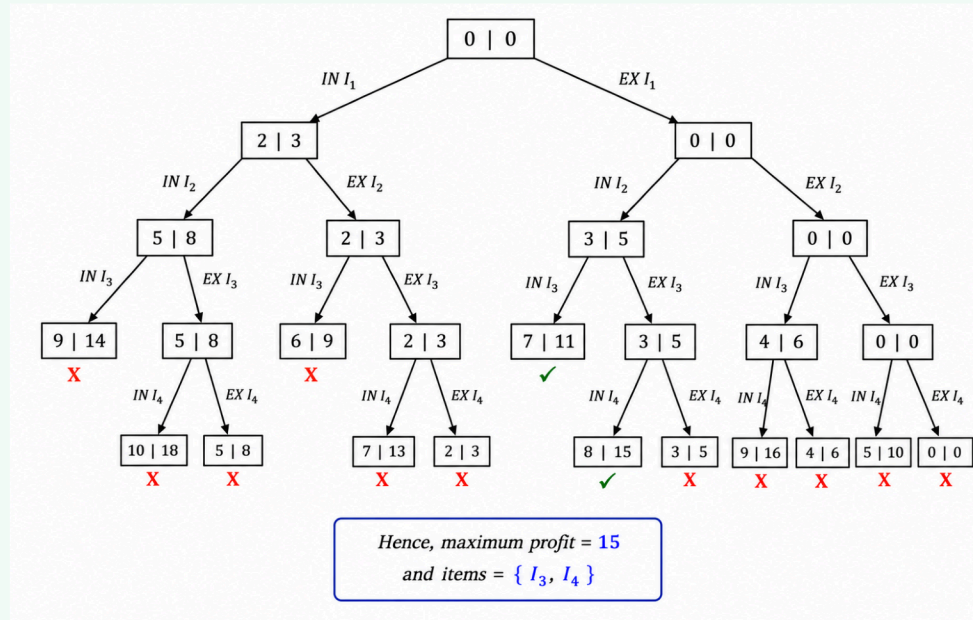
Hence, the total space complexity remains:

$$O(n)$$

Example 6.5.1

Example: Given 4 items with the following data: Items = {1,2,3,4}, $w = \{2, 3, 4, 5\}$, $p = \{3, 5, 6, 10\}$, $W = 8$. Find the maximum profit and the items chosen.

Solution:

**Question 6.2**

You have 4 items with the following weights and profits:

Items: 1, 2, 3, 4, Weights: 2, 3, 5, 7 and Profits: 3, 4, 8, 10

Knapsack capacity (W): 10

Find the maximum profit that can be obtained and which items should be included in the knapsack.